

Maritime CargoService: Sprint 0

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum - Università di Bologna

June 23, 2026

1. Introduction

Una compagnia di trasporto marittimo di container (d'ora in poi *la committente*) intende automatizzare le operazioni di carico dei container nella stiva della nave (d'ora in poi **hold**). A tal fine prevede di impiegare un robot a guida differenziale (Differential Drive Robot, d'ora in poi **cargorobot**).

L'obiettivo dello Sprint 0 è formalizzare in modo preciso e non ambiguo i requisiti forniti dalla committente:

The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer **retrylater**, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the **slots1-4** are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message '**Service working**', when it is all is going well
- the message '**Out of service**' if the *sonar sensor* measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the *sonar*).

Figure 1: Requisiti del committente

costruire un primo modello logico dei macro-componenti del sistema, evidenziare il *core business*, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali.

Ogni scelta è strettamente ancorata ai requisiti: il modello qui presentato serve a catturare il comportamento richiesto, senza anticipare decisioni progettuali o scelte implementative che verranno affrontate negli sprint successivi.

La traccia del progetto può essere scaricata dal seguente [link](#)^o

2. Motivazione dell'uso del linguaggio QAK

QAK è il linguaggio messo a disposizione dalla nostra software house (unibo.issLab) per modellare sistemi software distribuiti, particolarmente espressivo nel formalizzare il concetto di **attore autonomo** e di **messaggio**, riducendo di molto l'“Abstraction Gap” tra requisiti e modello di un sistema distribuito eterogeneo. La natura **reattiva** e **proattiva** di *cargoservice*, che deve rispondere a stimoli esterni e avviare autonomamente sequenze di azioni, è infatti catturata in modo naturale da un attore QAK, cosa che un POJO (Plain Old Java Object), componente passivo attivato da chiamate sincrone, non catturerebbe altrettanto bene. Dal modello QAK viene generato automaticamente codice Kotlin eseguibile, il che consente di disporre di un **primo prototipo**

osservabile già nello **Sprint 0**, prima ancora di scrivere una riga di logica applicativa. E' possibile trovare maggiori informazioni sul linguaggio QAK al seguente [link](#)^o

3. Requirements

3.1. cargorobot

Il cargorobot è l'entità software che dovrà governare un Differential Drive Robot (DDR): con questo termine si intende un tipo di robot mobile che per spostarsi usa due ruote indipendenti tra loro nei lati del suo "corpo". Il cargorobot si occuperà quindi di amministrare gli spostamenti del DDR in questione. Il suo compito principale è quello di fare in modo che il DDR trasporti i container in specifici slot. Per container si intende semplicemente un contenitore di forma, dimensione e contenuto non precisati.

Si ritiene opportuno rappresentare il cargorobot come un QAK Actor. Dovendo governare i movimenti del DDR all'interno della hold, necessita di un controllo autonomo per poter ricevere ed eseguire in modo asincrono le richieste di movimento.

3.2. hold

La hold rappresenta lo spazio virtuale all'interno del quale opera il cargorobot. Ha una forma piatta e rettangolare e al suo interno ospita:

- Dispositivo di Input/Output chiamato IOPort, questa è dotata di:
 - **Pushbutton**, premuto dal cliente per inviare una richiesta di caricamento di un container sul carico.
 - **Display**, utilizzato per mostrare la risposta alla richiesta e per mostrare lo stato attuale del blocco.
- 5 slots rappresentanti specifiche aree della hold nelle quali si conservano i container, di questi 4 sono normali (slot1—slot4) e uno è "speciale" (slot5) ed è

riservato per determinate funzionalità.

- Un sensore di rilevamento della distanza

Si ritiene opportuno che rappresentare la hold come QAK Actor sia ottimale, permettendo così di separare al meglio la logica dei dati dalla logica di business (responsabilità di cargoservice). Anche IOPort, nonostante l'attività di ricevere e mostrare messaggi del Display sia passiva, verrà rappresentato come un QAK Actor, dato che le funzionalità del pushbutton richiedono un funzionamento autonomo in grado di generare request al cargoservice.

3.2.1. sensore

Il Sensore è un dispositivo di rilevamento (un sonar) associato all'IOPort. Il suo compito è quello di rilevare la presenza di container da caricare e di comunicare costantemente le sue rilevazioni. La distanza sarà calcolata basandosi su una precisa area della hold chiamata `sensor_area`.

Si ritiene opportuno rappresentare il sensore come QAK Actor, essendo un'entità attiva che effettua e invia misurazioni in maniera completamente autonoma.

3.3. marker

Il marker è un dispositivo che si occupa di assegnare a un container, presente in quel preciso momento nello slot5, un determinato codice a barre. Dopodichè dovrà comunicare la fine dell'operazione al sistema.

Si ritiene opportuno modellare il marker come un QAK Actor.

3.4. cargoservice

- Il servizio da realizzare è **cargoservice**: il customer gli invia una richiesta di carico tramite il pushbutton dell'IOPort e cargoservice si occupa del funzionamento

generale dell'applicazione, in particolare dovrà soddisfare le seguenti indicazioni:

- Ricevere e interpretare i dati del sonar. In particolare, definendo una distanza D e un'altra D_{free} :
 - Se il sensore misura $D > D_{FREE}$ per ≥ 3 s dovrà mettere il sistema nello stato Out of service.
 - Se il sensore misura $D < D_{FREE}/2$ per ≥ 3 s significa che il sensore ha rilevato un container e il normale funzionamento può iniziare, quindi cargoservice

comanda al cargorobot di spostare il container **dall'IOPort** allo **slot5** (adibito al deposito temporaneo di un container prima della collocazione definitiva).

- Gestire il funzionamento del pushbutton, se viene premuto:
 - Se l'IOPort è occupata da un container o il sistema è *Out of service*, cargoservice risponde **retrylater**.
 - Se tutti gli slot1–slot4 (ciascuno capace di contenere un container) sono occupati, cargoservice **rifiuta** la richiesta.
 - Se l'IOPort è libera, il sistema è in servizio e almeno uno tra slot1–slot4 è libero, cargoservice entra nello stato *engaged*, riserva uno slot libero senza criteri di ottimizzazione e restituisce al customer il nome dello slot riservato.
- Gestire il display dell'IOPort, facendo sì che in ogni momento mostri:
 - lo **stato corrente della hold**.
 - il messaggio “**Service working**” durante il normale funzionamento.
 - Il messaggio “**Out of service**” se il sonar misura $D > D_{FREE}$ per ≥ 3 s.
- Inoltre deve controllare che:
 - Fintanto che il sistema è *engaged*, un **LED lampeggia**.
 - Dopo l'accettazione, il customer ha un tempo prefissato a 30 s per depositare il container nell'area del sonar. Se il customer non deposita entro il timeout, il sistema torna *disengaged*.
 - Il marker device etichetta il container in slot5 e **ne segnali il completamento**, dopo il quale cargoservice dovrà comandare al cargorobot di spostare il container **da slot5 allo slot riservato**.

Si ritiene opportuno rappresentare cargoservice come un QAK Actor, dovendo gestire il ciclo di business reagendo in tempo reale agli eventi del sonar e inviando comandi. Il LED invece, svolgendo solamente attività passive, sarà rappresentato da un POJO.

3.5. Domande aperte alla committente

Domanda al committente.

Posizione dell'IOPort nella mappa. I requisiti indicano che il cargorobot sposta il container *dall'IOPort a slot5*, lasciando intendere che l'IOPort sia una cella praticabile. Come si colloca nella griglia?

Domanda al committente.

Richieste concorrenti. Il sistema deve bufferizzare più richieste di carico contemporanee, oppure una nuova richiesta ricevuta mentre il sistema è *engaged* viene semplicemente rifiutata / respinta con *retrylater* senza accodamento? Dai requisiti si interpreta che le richieste **non siano bufferizzate**: se il sistema è occupato, la risposta è immediata (*retrylater* o *refused*) senza code di attesa.

Domanda al committente.

Liberazione degli slot e aggiornamento della disponibilità. I requisiti descrivono il processo di carico dei container negli slot1-4, ma non specificano come venga gestita la liberazione di uno slot già occupato. Possiamo assumere che lo svuotamento degli slot sia effettuato da sistemi o operatori esterni al nostro sistema? In tal caso, con quale meccanismo viene notificato a cargoservice che uno specifico slot è stato liberato e può tornare disponibile per future prenotazioni?

Domanda al committente.

Ripristino dello stato Service working. I requisiti specificano che il sistema entra nello stato *Out of service* quando il sonar rileva una distanza $D > D_{FREE}$ per almeno 3 secondi. Non è invece specificata la condizione per il ritorno allo stato *Service working*. Il sistema deve tornare operativo non appena il sonar rileva $D \leq D_{FREE}$ oppure è richiesto un ulteriore intervallo di stabilità prima del ripristino del servizio?

4. Requirement analysis

4.1. Core business

Il **core business** del sistema è la gestione del ciclo di carico di un container. La responsabilità della sequenza applicativa resta in **cargoservice**: il cargorobot è coinvolto per eseguire spostamenti richiesti dal servizio, non per decidere se una richiesta debba essere accettata, rifiutata o sospesa. La sequenza principale ricavata dai requisiti è:

1. Il customer preme il pushbutton dell'IOPort, quest'ultimo invia una Request a cargoservice.

2. cargoservice, interrogando hold, verifica le precondizioni (stato sistema, IOPort, disponibilità slot).
3. Se soddisfatte cargoservice imposta lo stato di sistema *engaged*, comunica alla hold di prenotare uno slot, invia al LED l'ordine di lampeggiare e notifica al customer.
4. cargoservice fa partire il timer interno di 30 s.
5. Se il sonar entro il timeout rileva un container, cargoservice comanda al cargorobot di spostarlo da IOPort a slot5.
6. Il marker device etichetta il container e segnala il completamento.
7. cargoservice comanda al cargorobot di spostare il container da slot5 allo slot riservato, reimpostando lo stato di sistema a *disengaged*.

4.2. Macro-componenti e natura software

COMPONENTE	RUOLO	STATO DI DISPONIBILITÀ	NATURA SOFTWARE
cargoservice	Orchestratore principale. Natura reattiva (richieste in arrivo) e proattiva (comandi al cargorobot, aggiornamenti display). Non può restare privo di controllo: da requisito deve rispondere e agire autonomamente.	Da sviluppare	QAK Actor
cargorobot	Entità che governa il DDR per la movimentazione fisica richiesta dal cargoservice. La scelta POJO vs microservizio è rimandata all'analisi: se fosse un POJO, il cargoservice gli cederebbe il controllo (trasferimento di controllo sincrono) e non potrebbe reagire ad altri eventi durante la movimentazione. Se invece fosse un microservizio, la comunicazione sarebbe asin-	Da chiarire / da sviluppare	QAK Actor

	crona e message-driven, con un disaccoppiamento molto maggiore. Forti motivazioni spingono verso la seconda opzione, ma la decisione è oggetto degli sprint successivi.		
IOPort	Interfaccia con il customer (pushbutton + display). Se realizzata come microservizio separato, il pushbutton come un button HTML e il display come textarea HTML), le responsabilità sono completamente disaccoppiate da cargoservice e i due possono essere sviluppati in parallelo; l'unica interfaccia condivisa è il messaggio <i>load_request</i> definito in questo sprint.	Dispositivo fisico fornito sw da sviluppare	QAK Actor
sonar	Sensore di distanza.	Dispositivo fisico fornito driver sw da sviluppare	QAK Actor
marker device	Etichettatura in slot5.	Dispositivo fisico simulato fornito sw da sviluppare	QAK Actor
LED	Indicatore stato <i>engaged</i> . Può essere fisico o virtuale. La scelta dipende	Fisico o virtuale sw da sviluppare	POJO

	dall'infrastruttura disponibile.		
hold	Struttura dati per lo stato della stiva (occupazione slot). Passivo.	Da sviluppare	QAK Actor

4.3. Formalizzazione dei messaggi QAK

La seguente formalizzazione non introduce ancora scelte di progetto, limitandosi ad elencare i messaggi necessari per esprimere i requisiti. **Request/Reply** viene usato quando il requisito prevede una risposta osservabile. ; **Event** viene invece tipicamente usato quando l'emettitore non sa a chi arriverà il messaggio e si “preoccupa” solamente di emettere informazioni.

4.3.1. customer / IOPort -> cargoservice

```
Request load_request : loadRequest(none)
Reply   load_accepted : loadAccepted(slotID)    for load_request
Reply   load_retrylater : loadRetryLater(none)    for load_request
Reply   load_refused   : loadRefused(none)       for load_request
```

La Request `load_request` rappresenta il momento nel quale viene effettuata una richiesta tramite il pushbutton dell'IOPort e le 3 Reply sono le relative risposte del cargoservice: nel caso la richiesta venisse accettata si consegna anche l'ID dello slot nel quale va posizionato il carico.

4.3.2. Sonar ->

```
Event sonar_distance : distance(D)
```

Questo evento rappresenta la comunicazione dei propri dati da parte del sonar al sistema. Probabilmente sarà cargoservice ad “ascoltare” questo evento.

4.3.3. marker device -> cargoservice

```
Dispatch marking_done : markingDone(containerID)
```

Messaggio di conferma a cargoservice di fine lavoro del marker.

4.3.4. cargoservice -> display / LED

```
Dispatch display_hold_state : holdState(state)
Dispatch display_status     : displayStatus(message)
Dispatch led_on              : ledOn(none)
Dispatch led_off             : ledOff(none)
```

Questi Dispatch rappresentano rispettivamente la “consegna” dei dati sullo status generale e sullo stato della hold da parte del cargoservice al display e le richieste di

accendere o spegnere il LED. Le richieste al LED avverranno in corrispondenza di eventuali cambi di stato (**engaged** o **disengaged**).

4.4. Contesti logici

CONTESTO	COMPONENTI E RESPONSABILITÀ
ctxCargoService	Contiene l'attore cargoservice. Nucleo del comportamento richiesto e punto di orchestrazione del ciclo di carico.
ctxCustomer	Raggruppa le entità dedicate all'interazione con l'utente: IO-Port (con display e pushbutton) e LED.
ctxDevices	Raggruppa i dispositivi presenti nella hold: sonar, hold e markerdevice.
ctxRobot	Raggruppa le entità legate al cargorobot e alla movimentazione richiesta.

4.5. Schema della hold

Legenda: H = HOME S = sonar/sensor area 1-4 = slot1-4
5 = slot5 I = IOPort X = ostacolo . = libero

```

      col0 col1 col2 col3 col4 col5 col6
riga0 [ H ][ S ][ . ][ . ][ . ][ . ][ . ]
riga1 [ . ][ 1 ][ X ][ X ][ 2 ][ . ][ . ]
riga2 [ . ][ . ][ . ][ . ][ X ][ 5 ][ . ]
riga3 [ . ][ 3 ][ X ][ X ][ 4 ][ . ][ . ]
riga4 [ . ][ . ][ . ][ . ][ . ][ . ][ . ]
riga5 [ I ][ X ][ X ][ X ][ X ][ X ][ X ]

```

Nota: La posizione esatta dell'IOPort e del sonar richiede conferma dalla committente. Lo schema è una prima interpretazione fedele alla figura allegata ai requisiti.

5. Test plan

I test funzionali verificano il comportamento osservabile del sistema rispetto ai requisiti, indipendentemente dall'implementazione.

SCENARIO	PRECONDIZIONI	RISULTATO ATTESO
Accettazione	disengaged , non OoS, IOPort libera, slot disponibile	load_accepted con slot riservato; stato engaged ; LED lampeggiante
Rifiuto (hold piena)	disengaged , non OoS, IOPort libera, slot1-4 tutti occupati	load_refused ; stato disengaged ; LED spento

Retrylater (OoS)	Sistema Out of service	load_retrylater ; stato immutato
Retrylater (IOPort occupata)	disengaged , non OoS, IOPort occupata da un container	load_retrylater ; stato disengaged
Timeout deposito	Sistema engaged ; customer non deposita entro il tempo prefissato	Sistema disengaged ; LED spento
Ciclo completo	disengaged , non OoS, IOPort libera, slot disponibile	Container nello slot riservato; display “ Service working ”; disengaged ; LED spento
Guasto sonar	Sistema operativo; sonar misura $D > D_{FREE}$ per ≥ 3 s	Sistema Out of service ; display “ Out of service ”
Rilevazione container	Sistema engaged ; sensor area vuota; sonar misura $D < D_{FREE}/2$ per ≥ 3 s	cargoservice avvia la movimentazione verso slot5

6. Project

Dall’analisi dei requisiti si propone, come ipotesi iniziale, una struttura a **tre sprint** con integrazione progressiva dei componenti. La suddivisione serve a organizzare il lavoro e i test, non a fissare decisioni architetturali definitive.

6.1. Sprint 1: Core business

Goal: realizzare il primo nucleo eseguibile di cargoservice con collaboratori simulati.

Funzionalità: ciclo di carico completo con collaboratori simulati, modello dello stato della hold, LED simulato, timeout.

6.2. Sprint 2: IOPort, display e sonar

Goal: rendere esplicita l’interazione con IOPort, display e sonar a livello software.

Funzionalità: IOPort come interfaccia web, sonar simulato (rilevazione container e malfunzionamento), aggiornamento display con stato e messaggi.

6.3. Sprint 3: Dispositivi fisici

Goal: integrare, dove disponibili, i dispositivi fisici o i servizi forniti e verificare il comportamento end-to-end.

Funzionalità: cargorobot e servizio DDR disponibile, sonar fisico, marker device fisico, LED fisico.

7. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340